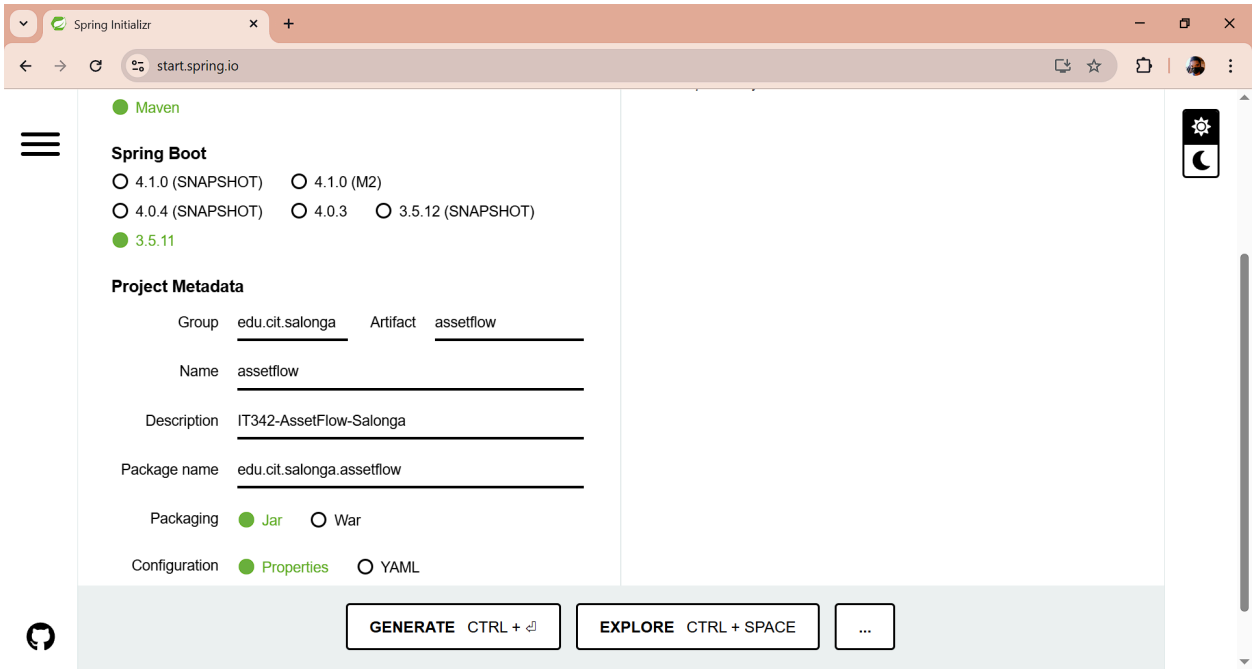
Salonga, Andre D. IT342-G1 March 7, 2026

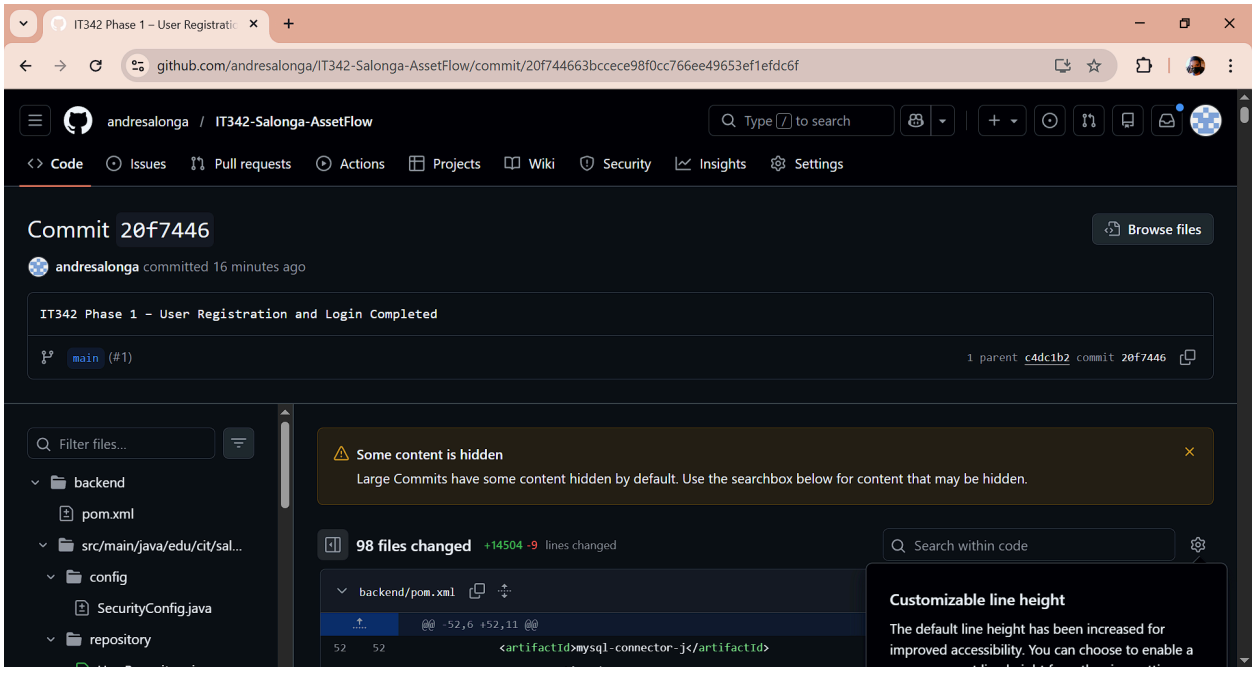
1. GitHub Repository Link: https://github.com/andresalonga/IT342-Salonga-AssetFlow.git

2. Backend Project Naming Convention (Spring Boot)



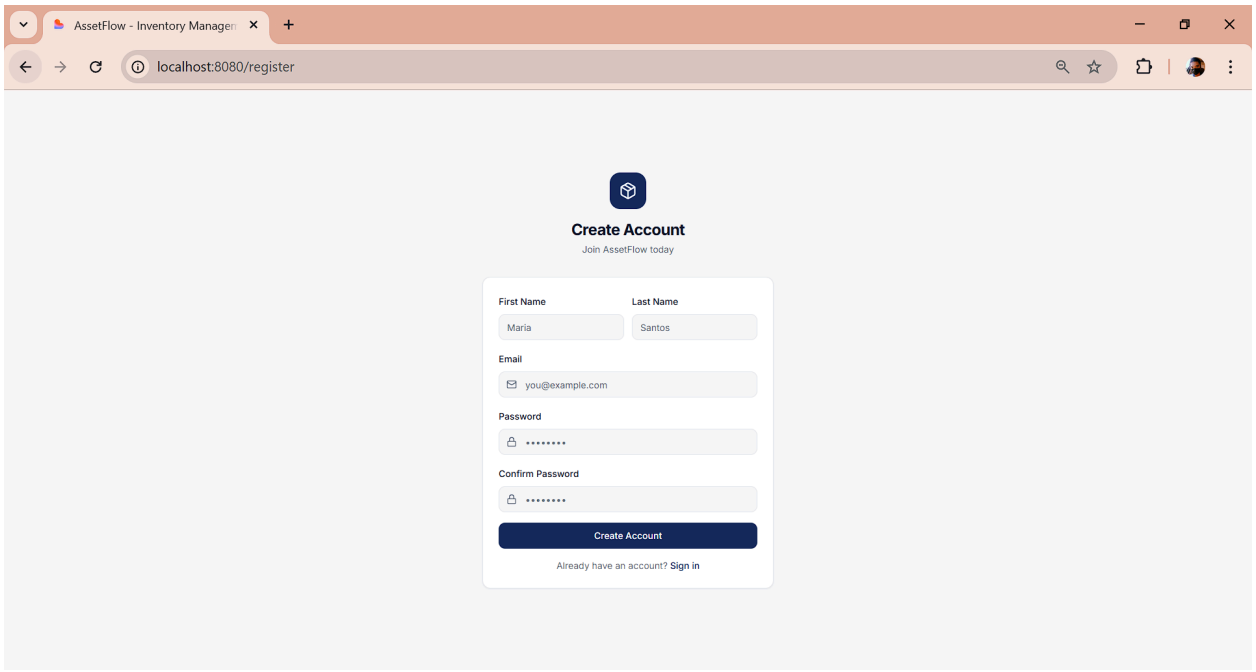
3. Final Commit for Phase 1

IT342 Phase 1 – User Registration and Login Completed:
https://github.com/andresalonga/IT342-Salonga-AssetFlow/commit/20f744663bccece98f0cc766ee49653ef1efdc6f

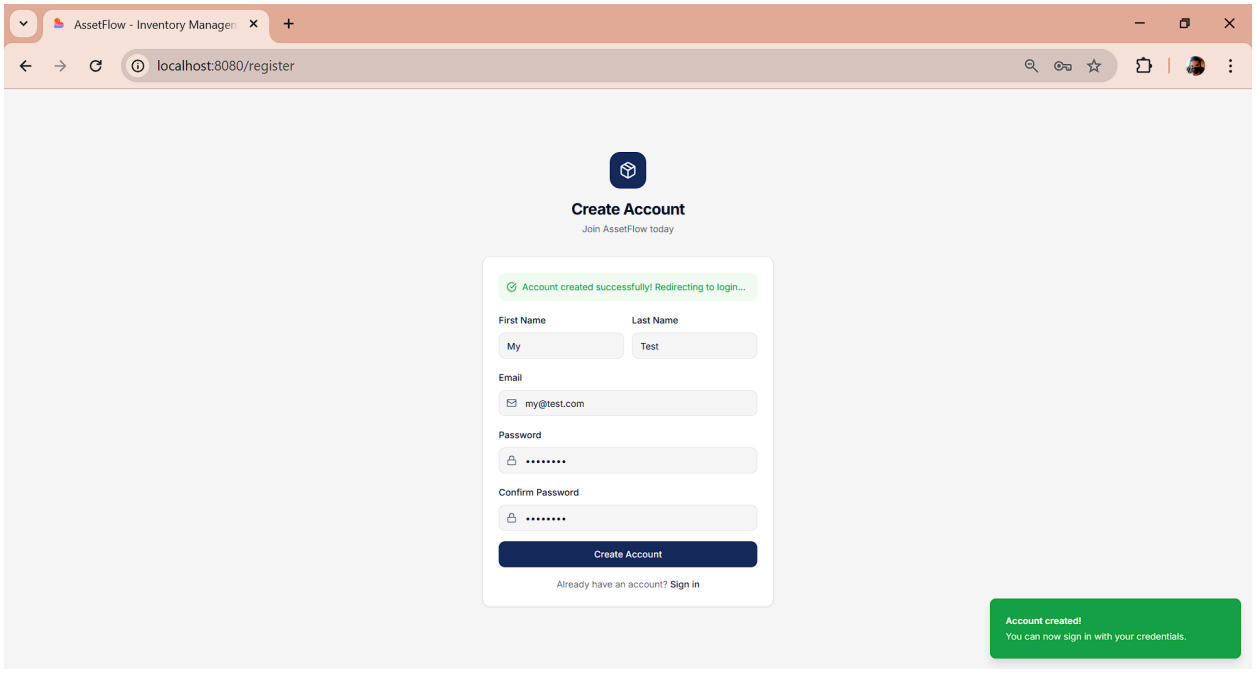


4. Screenshots of Implementation

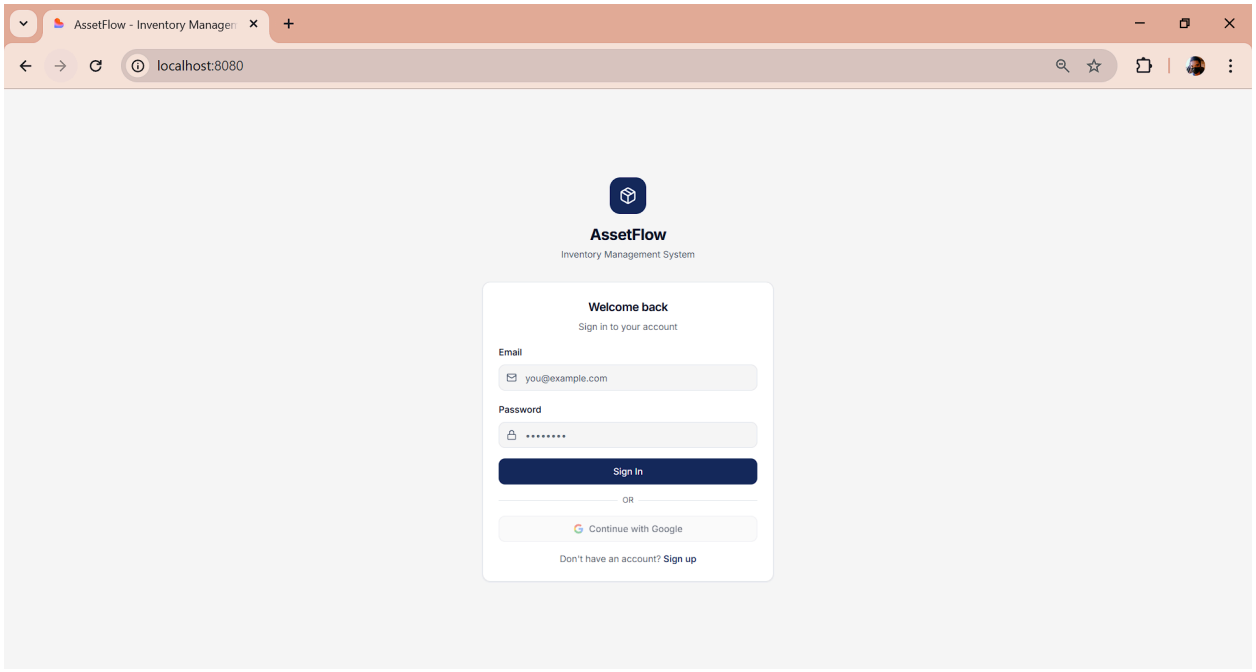
Registration page



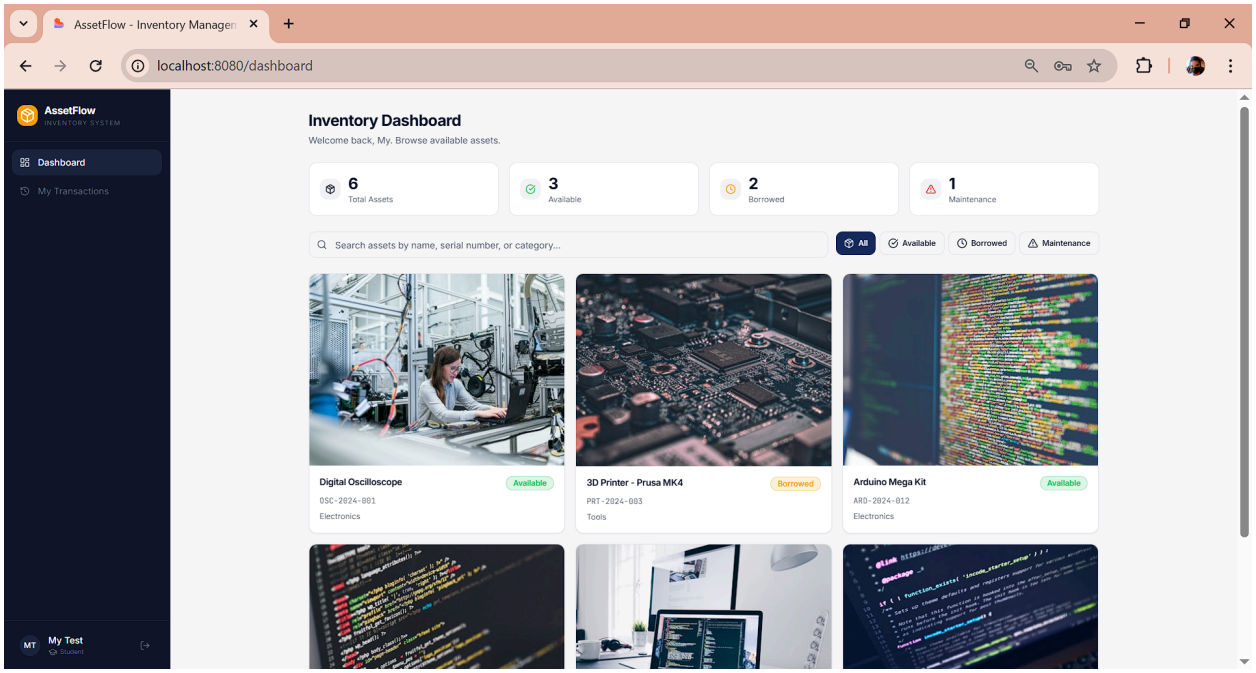
Successful user registration



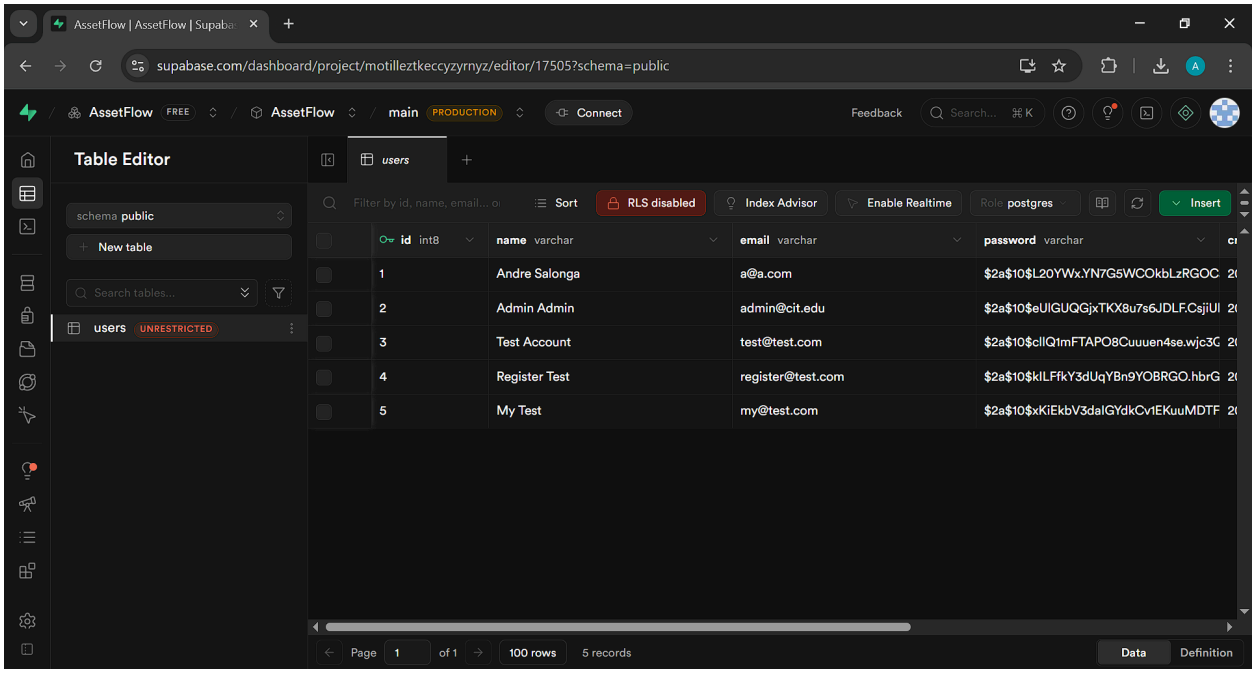
Login page



Successful login



Database record showing the registered user



5. Short Implementation Summary

User Registration

Registration Fields Used:

Field	Type	Validation
name	String	Required, Not blank
email	String	Required, Valid email format
password	String	Required, minimum 6 characters

The frontend combines firstName and lastName into a single name field before sending to the backend.

Validation Process:

- **Frontend (RegisterPage.tsx):**
  - All fields are required (checked via Object.values(form).some(v => !v))
  - Password and confirm password must match
  - Password must be at least 6 characters
  - Email format validated via HTML5 input type
- **Backend (RegisterRequest.java):**
  - Uses Jakarta Bean Validation (@NotBlank, @Email, @Size)
  - Server-side validation ensures data integrity even if client validation is bypassed

Duplicate Account Prevention:

- The UserRepository.existsByEmail() method checks if the email already exists before registration
- The AuthService.register() method returns an error response: "Email already registered" if a duplicate is found
- The email column in the database has a unique constraint (@Column(unique = true) in User.java)

Secure Password Storage:

- Passwords are hashed using BCrypt (via BCryptPasswordEncoder)
- The raw password is never stored in the database
- BCrypt includes automatic salt generation for each password

---

User Login

Login Credentials Used:

Field	Type	Validation

email	String	Required, Valid email format
password	String	Required

Verification Process:

1. Frontend sends email and password to POST /api/auth/login
2. AuthService.login() retrieves the user by email from the database
3. If user not found → returns "User not found" error
4. If found → uses encoder.matches() to compare the plaintext password against the stored BCrypt hash
5. If password doesn't match → returns "Invalid credentials" error

Post-Login Behavior:

1. On successful authentication, a JWT (JSON Web Token) is generated via JwtUtil.generateToken()
2. The token is returned to the client along with user info
3. Frontend stores the token in localStorage via saveToken()
4. User data is stored in localStorage for session persistence
5. User is redirected to /dashboard
6. The JWT token is included in subsequent API requests via the Authorization: Bearer <token> header

---

Database Table

Table: users

Column	Type	Constraints

id	BIGINT	Primary Key, Auto-increment
name	VARCHAR(255)	Not null
email	VARCHAR(255)	Unique, Not null
password	VARCHAR(255)	Not null (BCrypt hash)
role	VARCHAR(255)	Default: 'USER'
created_at	DATETIME	Auto-generated

API Endpoints

Method	Endpoint	Description

POST	/api/auth/register	Register new user
POST	/api/auth/login	Authenticate user, get JWT
GET	/api/auth/me	Get current authenticated user
POST	/api/auth/logout	Logout (client removes token)